

La théorie des graphes au service de l'urbanisme

Lou Toubiana et Léon Hogenmuller

7 novembre 2025

Sommaire

1	Explication de notre problématique	2
2	Terminologie	2
2.1	Définitions	2
2.2	Modélisation en théorie des graphes	3
2.3	Conventions et notations	3
2.4	Formulation du problème	4
3	État de l'art	4
4	Programme linéaire	5
4.1	Paramètres et variables de décision	5
4.2	Minimiser la Somme	6
4.3	Illustration concrète sur un exemple	6
5	Heuristique	8
5.1	Quelques lemmes dûs à Chung et al.	9
5.2	Description de l'algorithme	9
5.3	Pseudocode de Chung, Garey et Tarjan	10
5.4	Exemples concrets	10
5.4.1	Exemple I : Un graphe orientable	11
5.4.2	Exemple II : Un graphe non orientable	11
5.4.3	Exemple III : Encore un graphe non orientable	11
6	Application à notre problème	12
6.1	Rendre G orientable	12
6.2	Prétraitement	12
6.2.1	Orientation des strong bridges	12
6.2.2	Choix stratégique des couples (s,t)	13
7	Méta-heuristique	14
8	Résultats	16
8.1	Génération des données	16
8.2	Tests sur le Prétraitement	17
9	Bilan technique et travaux futurs	19
	Annexe C : Algorithme DFS orientation	21
	Annexe D : Minimiser le Max	22

1 Explication de notre problématique

Aujourd'hui, avec la transition écologique, l'augmentation du nombre de cycliste et donc la multiplication des modes de transport, de nouvelles questions se posent. En effet, les villes doivent repenser leur réseau de circulation afin de notamment améliorer la sécurité de tous les usagers. Ces nouvelles préoccupations passent par le fait de transformer certaines rues étroites, et donc jusqu'à présent en double sens, en sens unique pour laisser davantage de place aux piétons, aux cyclistes ou même pour plus de verdure en ville.

En règle générale, il s'agit de petites modifications à l'échelle d'un ou deux quartiers. Néanmoins, elles peuvent malheureusement parfois perturber l'accessibilité de la ville dans son ensemble. Ainsi, avant d'instaurer le moindre sens unique, il est primordial de s'assurer que chaque point de la ville reste accessible depuis n'importe quel autre sans que cela n'occasionne de détours trop importants.

Le défi consiste donc à concilier interventions locales et avec la cohérence globale. Transformer certains sens de circulation pour répondre à des besoins d'aménagement ne doit pas compromettre la fluidité générale du trafic.

Ce problème peut être formalisé à l'aide d'un graphe mixte, représentant le réseau : les rues déjà orientées (sens unique imposé) correspondent aux arcs, tandis que les rues encore en double sens sont modélisées par des arêtes non orientées. On doit donc choisir un sens de circulation pour les arêtes tout en garantissant que l'on puisse toujours aller d'un point quelconque de la ville à un autre, et en limitant les augmentations de temps de trajet. Cette question se pose donc à l'intersection entre accessibilité, efficacité des trajets, et contraintes d'urbanisme.

Par la suite, pour résoudre ce problème nous allons utiliser différents outils mathématiques et techniques d'optimisations. On introduira d'abord le vocabulaire mathématiques et les notations utilisés pour modéliser notre problème. Puis, nous présenterons qu'est-ce qui a déjà été fait pour résoudre notre problème dans la littérature scientifique. Ensuite, nous présenterons un modèle mathématiques permettant de résoudre le problème. Puis, nous décrirons une approche heuristique pour fournir rapidement une solution à notre problème mais sans garantie sur la valeur de la solution et comment nous l'appliquons à notre problématique, notamment avec la mise en place de prétraitement. Enfin, nous analyserons des résultats expérimentaux.

2 Terminologie

Afin d'éviter toute ambiguïté on fixe ici les notations, définitions et conventions que nous utiliserons tout au long de ce document.

2.1 Définitions

Définition 1. Une **arête** est un couple de sommets noté $\{i, j\}$ sans relation d'ordre, qui relie deux sommets i et j .

Définition 2. Un **arc** est un couple de sommets noté (i, j) , avec une relation d'ordre, qui représente une liaison dirigée de i vers j .

Définition 3. Un **graphe non orienté** est un graphe $G = (V, E)$ avec V un ensemble de sommets et E un ensemble d'arêtes.

Définition 4. Un **graphe orienté** est un graphe $G = (V, A)$ avec V un ensemble de sommets et A un ensemble d'arcs.

Définition 5. Un **graphe mixte** est un graphe $G = (V, E, U)$ avec V un ensemble de sommet, E un ensemble d'arêtes et U un ensemble d'arcs.

Définition 6. Le **diamètre d'un graphe** est le plus grand des plus courts chemins entre deux sommets d'un graphe.

Définition 7. Une **chaîne** est une suite d'arêtes, $\{i, j\}, \{j, k\}, \{k, m\}, \dots$, dont le sommet de fin de l'arête e_n et le sommet de début de l'arête e_{n+1} est le même.

Définition 8. Un **chemin** est une suite d'arcs, $(i, j), (j, k), (k, m), \dots$, dont le sommet de fin de l'arc e_n et le sommet de début de l'arc e_{n+1} est le même.

Définition 9. Un graphe est **connexe** s'il existe une chaîne entre chaque paire de sommets (en oubliant l'orientation on peut aller de n'importe quel sommet à un autre).

Définition 10. Un graphe orienté est **fortement connexe** s'il existe un chemin entre chaque paire ordonnée de sommets (i, j) .

Définition 11. Une **composante connexe** est un sous-graphe maximal connexe.

Définition 12. Une **composante fortement connexe** est un sous-graphe orienté maximal fortement connexe.

Définition 13. Une **composante 2-connexe** est un sous-graphe connexe qui reste connexe même si on lui enlève une arête quelconque.

Définition 14. Un **bridge** est une arête dont la suppression augmente le nombre de composantes connexes.

Définition 15. Un **strong bridge** est un arc dont la suppression augmente le nombre de composantes fortement connexes.

Définition 16. Pour un sommet i :

- $\Gamma(i)$ désigne l'ensemble des voisins de i , sans prendre en compte l'orientation des arêtes ;
- $\Gamma^+(i)$ est l'ensemble des successeurs de i (tels que $(i, j) \in A$) ;
- $\Gamma^-(i)$ est l'ensemble des prédécesseurs de i (tels que $(j, i) \in A$).

Définition 17. Un graphe $G = (V, E)$ non-orienté ou mixte est dit **orientable** en un graphe fortement connexe s'il est possible d'orienter toutes ses arêtes de manière à ce que le graphe entièrement orienté obtenu soit fortement connexe. On parle d'**orientabilité**.

2.2 Modélisation en théorie des graphes

On va représenter le réseau urbain par un graphe mixte, c'est-à-dire un graphe composé :

1. de sommets : ils correspondent aux intersections des rues.
2. d'arcs : ils correspondent ici aux rues dont on ne veut pas modifier le sens.
3. d'arêtes : elles correspondent ici aux rues en double sens que l'on souhaite passer en sens unique.

On s'intéresse plus précisément au problème suivant : orienter les arêtes de manière à ce que le graphe résultant soit fortement connexe.

Ce type de problème est crucial dans un contexte urbain : il garantit que toute zone reste accessible, sans devoir contourner le réseau de manière excessive.

Cependant, on doit aussi prendre en compte l'impact que l'orientation des arêtes aura sur la distance entre chaque sommet du graphe. On voudrait que celle-ci augmente le moins possible.

2.3 Conventions et notations

Dans notre contexte, nous partons d'un graphe mixte $G = (V, E, U)$, où :

- L'ensemble V correspond à l'ensemble des sommets (intersections, points du réseau),
- L'ensemble E correspond à l'ensemble des arêtes (rues encore en double sens),
- L'ensemble U correspond à l'ensemble des arcs (rues dont on ne souhaite pas modifier le sens de circulation).

On note $\vec{G} = (V, A)$ la solution de notre problème.

— L'ensemble $A = U \cup \vec{E}$, correspond à l'ensemble des arcs de la solution.

On introduit l'ensemble P correspondant à l'ensemble des paires origine destination -OD- pour lesquelles un chemin allant de l'origine à la destination doit exister : $P = \{(s, t) \mid (s, t) \in V^2, s \neq t\}$

On peut voir le récapitulatif de toutes les notations dans la **Table 1**.

Symbole	Type	Description
G	Graphe	Le graphe mixte
$\vec{G}$	Graphe	G orienté (solution du problème)
V	Ensemble	Ensemble de sommets de G
E	Ensemble	Ensemble d'arêtes de G
$\vec{E}$	Ensemble	Ensemble d'arcs correspondant à l'orientation des arêtes
U	Ensemble	Ensemble d'arcs de G
A	Ensemble	$U \cup \vec{E}$
P	Ensemble	Ensemble des paires OD (s, t)

TABLE 1 – Notations utilisées dans la modélisation

2.4 Formulation du problème

L'objectif est donc d'orienter les arêtes E pour obtenir un graphe totalement orienté $\vec{G} = (V, A)$ fortement connexe, tout en minimisant la longueur totale des trajets.

3 État de l'art

Le problème de rendre un graphe non orienté, orienté et fortement connexe n'est pas une nouvelle question en théorie des graphes. Cependant, la majorité des recherches ont été menées sur des graphes non-orientés et non sur des graphes partiellement orientés.

L'une des premières réponses est donnée par le théorème de Robbins, formulé en 1939 [7], qui établit que :

Théorème 1. *Un graphe non orienté G admet une orientation fortement connexe si et seulement si G est connexe et ne contient aucun bridge, donc si G est 2-connexe.*

Ce résultat a constitué un point de départ pour l'étude des orientations fortes dans les graphes car il démontre que cela est possible et en temps linéaire.

Le théorème étendu de Robbins par Boesch et Tindell [1] permet de généraliser le théorème précédent aux graphes mixtes — c'est-à-dire contenant à la fois des arêtes non orientées et des arcs déjà orientés. Leur résultat est le suivant :

Théorème 2. *Un graphe mixte $G = (V, E, U)$ admet une orientation fortement connexe si et seulement si G est fortement connexe et le sous-graphe $G' = (V, E)$ est sans bridge.*

Ce théorème fournit donc une condition nécessaire et suffisante d'orientabilité dans le cas mixte, ce qui est exactement le cadre dans lequel nous nous plaçons pour modéliser un réseau routier partiellement orienté. Il garantit l'existence d'une orientation, mais ne donne pas de méthode constructive pour la trouver, ni ne prend en compte des critères supplémentaires comme les distances entre les sommets ou le diamètre.

L'orientation forte d'un graphe n'est qu'un aspect du problème. Dans des situations réelles, comme l'organisation d'un réseau routier, la qualité des chemins (et notamment leur longueur) est cruciale. Chvátal et Thomassen [3] ont étudié le problème de l'augmentation du diamètre après orientation. Ils montrent que certaines orientations peuvent augmenter considérablement les distances, et que la recherche d'une orientation optimale (au sens du diamètre ou de la somme des distances) est, dans de nombreux cas, difficile, voire NP-difficile.

Plus récemment, de Vries [10] s'est intéressée à différentes variantes du problème, en étudiant plusieurs formulations d'optimisation et en développant des approches exactes et heuristiques. Dans un contexte plus applicatif, Conte et al. [4] proposent une méthode basée sur l'énumération de toutes les orientations fortes d'un graphe, afin de sélectionner celle qui satisfait certains critères de qualité.

Pour pallier la complexité du problème, des méthodes heuristiques sont souvent utilisées. Par exemple, l'algorithme de Chung et al [2] peut servir de point de départ, d'heuristique, pour une approche par recherche tabou. La recherche tabou, proposée par Glover [5], est une méthode de métaheuristique puissante pour explorer efficacement l'espace des solutions tout en évitant les cycles de recherche. Elle est souvent utilisée dans des problèmes combinatoires complexes où une solution exacte est trop coûteuse à obtenir.

Ces techniques sont particulièrement intéressantes dans un contexte comme le nôtre, où la recherche d'une orientation forte doit s'accompagner de contraintes supplémentaires (distances, accessibilité, etc.) liées aux enjeux de mobilité urbaine.

Dans la section suivante nous allons utiliser la programmation linéaire en nombres entiers pour modéliser notre problème et à l'aide d'un solveur obtenir une solution exacte (optimale).

4 Programme linéaire

Définition 18. *Un **programme linéaire (PL)** est un modèle mathématique qui permet de modéliser un problème, représenté par une **fonction objectif** qu'il faut optimiser (minimiser ou maximiser), tout en respectant un certain nombre de **contraintes**. On représente nos choix possibles par des **variables de décision** que l'on cherche à déterminer afin d'optimiser un objectif. Ce sont les inconnues du modèle, dont la valeur sera choisie par l'algorithme pour respecter les contraintes imposées tout en atteignant le meilleur résultat possible selon la fonction objectif.*

Exemple 1. *On cherche par exemple à maximiser un profit en vendant un produit, tout en respectant des limites précises, des contraintes : comme un budget, une capacité, ou un nombre de ressources. La fonction objectif correspond ainsi, à la maximisation du profit, les variables de décision correspondent au nombre de produits vendus/produits, et les contraintes sont le budget, les ressources etc...*

Dans un PLNE les expressions de la fonction objectif et des contraintes doivent être exprimés par des fonctions linéaires des variables de décision (sans exposant, multiplication de plusieurs variables de décision...). Si les variables de décision ou une parties sont des nombres entiers, on parle alors de programmation linéaire en nombres entiers (PLNE).

On peut donc représenter notre problème à l'aide d'un PLNE. Il y a deux fonctions objectifs raisonnables dans ce problème. Premièrement, la minimisation de la somme des plus courts chemins. Deuxièmement, la minimisation du diamètre. Nous avons décidé de travailler avec la première fonction objectif. Nous nous sommes inspirés du PLNE présenté dans [10]

4.1 Paramètres et variables de décision

Nous allons utiliser plusieurs variables et paramètres dans ce modèle. Les variables de décision utilisées ici sont :

- La variable $u_{ij} \in \{0, 1\}$ indique l'orientation choisie pour chaque arête non orientée $\{i, j\}$.
- La variable $x_{ij}^{st} \in \{0, 1\}$ vaut 1 si l'arc (i, j) est utilisé dans le chemin de s vers t .

Pour ce qui est des paramètres on a :

- Le paramètre λ_{st} correspond à la valeur du plus court chemin de s à t dans G avant modification.
- Le paramètre d_{ij} correspond au poids de l'arc allant de i à j .

D'autre part, pour simplifier les notations μ_{st} correspond à la valeur du plus court chemin de s à t dans $\vec{G}$.

On peut voir le récapitulatif de ces notations dans la **Table 2**.

Enfin, pour simplifier l'écriture du PLNE on notera $A' = U \cup \{(i, j), (j, i) \mid \forall \{i, j\} \in E\}$

Symbole	Type	Description
λ_{st}	Paramètre	Distance du plus court chemin entre s et t avant modification
μ_{st}	Notation	Distance du plus court chemin entre s et t après orientation
d_{ij}	Paramètre	Distance de l'arête/arc entre i et j
u_{ij}	Variable de décision	Indique si l'arête entre i et j est orientée de i vers j ou non
x_{ij}^{st}	Variable de décision	indique si l'arc de i à j est utilisé pour le chemin de s à t

TABLE 2 – Variables et paramètres utilisés dans le PLNE.

4.2 Minimiser la Somme

Ici, nous allons minimiser la somme des plus courts chemins entre toutes les paires de sommets de P .

On a donc :

$$\begin{cases}
 \min z = \sum_{st \in P} \sum_{(i,j) \in A'} d_{ij} \cdot x_{ij}^{st} & (1) \\
 \text{s.c.} \\
 \sum_{j \in \Gamma^+(i)} x_{ij}^{st} - \sum_{j \in \Gamma^-(i)} x_{ji}^{st} = 0 & \forall i \in V, \forall st \in P, s \neq i, t \neq i & (2) \\
 \sum_{j \in \Gamma^+(s)} x_{sj}^{st} - \sum_{j \in \Gamma^-(s)} x_{js}^{st} = 1 & \forall st \in P & (3) \\
 \sum_{j \in \Gamma^+(t)} x_{tj}^{st} - \sum_{j \in \Gamma^-(t)} x_{jt}^{st} = -1 & \forall st \in P & (4) \\
 x_{ij}^{st} \leq u_{ij} & \forall \{i, j\} \in E, st \in P & (5) \\
 u_{ij} + u_{ji} = 1 & \forall \{i, j\} \in E & (6) \\
 x_{ij}^{st} \in [0; 1] & \forall (i, j) \in A' & (7) \\
 u_{ij} \in \{0; 1\} & \forall \{i, j\} \in E & (8)
 \end{cases}$$

Dans ce modèle, l'objectif (1) consiste à minimiser la somme des distances associées aux plus courts chemins entre toutes les paires de sommets $(s, t) \in P$ après orientation du graphe. Les contraintes (2) à (4) sont des contraintes de conservation de flux :

pour le plus court chemin d'une paire OD s, t tout ce qui rentre dans un sommet doit en sortir sauf pour les extrémités. Cela permet de garantir que pour chaque paire s, t il y a au moins un chemin.

La contrainte (5) lie les variables de flux x_{ij}^{st} aux décisions d'orientation u_{ij} : elle garantit qu'un flux ne peut emprunter un arc (i, j) que si celui-ci a bien été orienté dans ce sens.

La contrainte (6) impose qu'une unique orientation soit choisie pour chaque arête du graphe initial : pour chaque paire $\{i, j\} \in E$, soit (i, j) , soit (j, i) est activé, mais jamais les deux.

Les contraintes (7) et (8) définissent les domaines des variables :

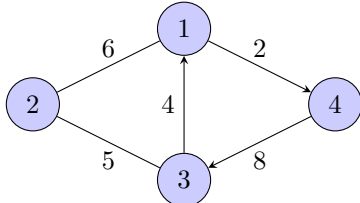
- Les variables x_{ij}^{st} sont continues dans l'intervalle $[0, 1]$ car elles modélisent un flux ;
- Les variables u_{ij} sont binaires et représentent les choix d'orientation.

Enfin, bien que les variables x_{ij}^{st} soient relaxées en variables continues, le modèle reste correct : comme les u_{ij} sont binaires et contraignent la direction autorisée pour chaque arc, les solutions optimales pour les flux s'avèrent de toute façon entières dans ce cadre (0 ou 1).

Pour bien comprendre le programme linéaire nous allons le décrire sur un exemple.

4.3 Illustration concrète sur un exemple

Voici un graphe mixte G que nous allons orienter complètement :



Nous allons appliquer le programme linéaire avec les quatre couples : $(2,4)$, $(4,2)$, $(1,3)$ et $(3,1)$. On définit l'ensemble $P = \{(2,4), (4,2), (1,3), (3,1)\}$.

Nous avons alors $E = \{\{1,2\}, \{2,3\}\}$. Enfin, l'ensemble $A' = \{(1,4), (4,3), (1,3), (2,3), (3,2), (1,2), (2,1)\}$

Nous allons maintenant expliciter les contraintes du PLNE avec les couples OD introduits dans P .

Pour la contrainte (2) on a :

$$\begin{aligned} \sum_{j \in \Gamma^+(i)} x_{ji}^{2,4} - \sum_{j \in \Gamma^-(i)} x_{ji}^{2,4} &= 0 \quad \forall i \in \llbracket 1, 4 \rrbracket \\ \sum_{j \in \Gamma^+(i)} x_{ji}^{4,2} - \sum_{j \in \Gamma^-(i)} x_{ji}^{4,2} &= 0 \quad \forall i \in \llbracket 1, 4 \rrbracket \\ \sum_{j \in \Gamma^+(i)} x_{ji}^{1,3} - \sum_{j \in \Gamma^-(i)} x_{ji}^{1,3} &= 0 \quad \forall i \in \llbracket 1, 4 \rrbracket \\ \sum_{j \in \Gamma^+(i)} x_{ji}^{3,1} - \sum_{j \in \Gamma^-(i)} x_{ji}^{3,1} &= 0 \quad \forall i \in \llbracket 1, 4 \rrbracket \end{aligned}$$

Pour la contrainte (3) on a :

$$\begin{aligned} \sum_{j \in \Gamma^+(2)} x_{2j}^{2,4} - \sum_{j \in \Gamma^-(2)} x_{2j}^{2,4} &= 1 \\ \sum_{j \in \Gamma^+(4)} x_{4j}^{4,2} - \sum_{j \in \Gamma^-(4)} x_{4j}^{4,2} &= 1 \\ \sum_{j \in \Gamma^+(1)} x_{1j}^{1,3} - \sum_{j \in \Gamma^-(1)} x_{1j}^{1,3} &= 1 \\ \sum_{j \in \Gamma^+(3)} x_{3j}^{3,1} - \sum_{j \in \Gamma^-(3)} x_{3j}^{3,1} &= 1 \end{aligned}$$

Pour la contrainte (4) on a :

$$\begin{aligned} \sum_{j \in \Gamma^+(4)} x_{4j}^{2,4} - \sum_{j \in \Gamma^-(4)} x_{4j}^{2,4} &= -1 \\ \sum_{j \in \Gamma^+(2)} x_{2j}^{4,2} - \sum_{j \in \Gamma^-(2)} x_{2j}^{4,2} &= -1 \\ \sum_{j \in \Gamma^+(3)} x_{3j}^{1,3} - \sum_{j \in \Gamma^-(3)} x_{3j}^{1,3} &= -1 \\ \sum_{j \in \Gamma^+(1)} x_{1j}^{3,1} - \sum_{j \in \Gamma^-(1)} x_{1j}^{3,1} &= -1 \end{aligned}$$

Pour la contrainte (5) on a :

$$\begin{aligned} x_{2,3}^{2,4} &\leq u_{2,3} \\ x_{3,2}^{2,4} &\leq u_{3,2} \\ x_{1,2}^{2,4} &\leq u_{1,2} \\ x_{2,1}^{2,4} &\leq u_{2,1} \end{aligned}$$

$$\begin{aligned} x_{2,3}^{4,2} &\leq u_{2,3} \\ x_{3,2}^{4,2} &\leq u_{3,2} \\ x_{1,2}^{4,2} &\leq u_{1,2} \\ x_{2,1}^{4,2} &\leq u_{2,1} \end{aligned}$$

$$\begin{aligned} x_{2,3}^{1,3} &\leq u_{2,3} \\ x_{3,2}^{1,3} &\leq u_{3,2} \\ x_{1,2}^{1,3} &\leq u_{1,2} \\ x_{2,1}^{1,3} &\leq u_{2,1} \end{aligned}$$

$$\begin{aligned} x_{2,3}^{3,1} &\leq u_{2,3} \\ x_{3,2}^{3,1} &\leq u_{3,2} \\ x_{1,2}^{3,1} &\leq u_{1,2} \\ x_{2,1}^{3,1} &\leq u_{2,1} \end{aligned}$$

Pour la contrainte (6) on a :

$$\begin{aligned} u_{2,3} + u_{3,2} &= 1 \\ u_{2,4} + u_{4,2} &= 1 \end{aligned}$$

Pour les contraintes de domaine (7) et (8) :

$$\begin{aligned} \forall \{i, j\} \in E, 0 \leq x_{ij}^{2,4} \leq 1 \quad u_{2,3} &\in \{0, 1\} & \forall \{i, j\} \in E, 0 \leq x_{ij}^{4,2} \leq 1 \quad u_{3,2} &\in \{0, 1\} \\ \forall \{i, j\} \in E, 0 \leq x_{ij}^{1,3} \leq 1 \quad u_{2,4} &\in \{0, 1\} & \forall \{i, j\} \in E, 0 \leq x_{ij}^{3,1} \leq 1 \quad u_{4,2} &\in \{0, 1\} \end{aligned}$$

Comme dans G il n'y a que deux arêtes nous avons 4 possibilités d'orientations :

1. $u_{2,1} = 1$ et $u_{3,2} = 1$
2. $u_{1,2} = 1$ et $u_{2,3} = 1$
3. $u_{2,1} = 1$ et $u_{2,3} = 1$
4. $u_{1,2} = 1$ et $u_{3,2} = 1$

Les solutions numéro 3 et 4 donnent des graphes qui ne sont pas fortement connexes. Il n'existe pas de chemin 3 vers 2 dans la solution numéro 3 et dans la numéro 4, il n'y a pas de chemin de 2 vers 3. Nous nous retrouvons donc avec seulement 2 possibilités.

Pour la fonction objectif on a :

$$\min \sum_{st \in P} \sum_{(i,j) \in A'} d_{ij} \cdot x_{ij}^{st} = \min \sum_{st \in P} \mu_{st}$$

$$u_{2,1} = 1 \text{ et } u_{3,2} = 1 :$$

$$\mu_{2,4} = 8, \mu_{4,2} = 13, \mu_{1,3} = 10 \text{ et } \mu_{3,1} = 4,$$

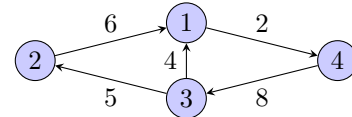
$$\text{on a } \sum_{st \in P} \mu_{st} = 35$$

$$u_{1,2} = 1 \text{ et } u_{2,3} = 1 :$$

$$\mu_{2,4} = 11, \mu_{4,2} = 18, \mu_{1,3} = 10 \text{ et } \mu_{3,1} = 4,$$

$$\text{on a } \sum_{st \in P} \mu_{st} = 43$$

Après résolution du programme linéaire, on voit que la meilleure solution est la numéro 1. On obtient le graphe suivant :



Cependant, si on ajoute dans P les paires OD (1,2), (2,1), (2,3) et (3,2) la solution optimale change.

pour la solution 1 on a :

$$\mu_{1,2} = 2 + 8 + 5 = 15$$

$$\mu_{2,3} = 6 + 2 + 8 = 16$$

$$\sum_{st \in P} \mu_{st} = 35 + 15 + 6 + 16 + 5 = 77$$

$$\mu_{2,1} = 6$$

$$\mu_{3,2} = 5$$

pour la solution 2 on a :

$$\mu_{1,2} = 6$$

$$\mu_{2,3} = 5$$

$$\sum_{st \in P} \mu_{st} = 43 + 6 + 9 + 5 + 10 = 73$$

$$\mu_{2,1} = 5 + 4 = 9$$

$$\mu_{3,2} = 4 + 6 = 10$$

Donc la solution optimale devient : $u_{1,2} = 1$ et $u_{2,3} = 1$.

On remarquera que quelle que soit la solution choisie les chemins (3,1),(1,3),(4,3) et (3,4) ne seront

jamais impactés, car leur plus courts chemins passent pas des arcs et non des arêtes. Ces couples ne sont donc pas pertinents lorsqu'on cherche une solution optimale.

La résolution du PLNE peut prendre du temps sur de grandes instances notamment, comme nous le verrons dans la section Résultats, car la combinatoire est importante. Donc dans la section suivante nous allons introduire un algorithme heuristique qui permet d'obtenir une solution réalisable (fortement connexe) sans garantie sur sa valeur (z).

5 Heuristique

Dans cette partie on cherche à implémenter un algorithme qui nous donne une orientation de G sans pour autant qu'elle soit optimisée.

L'article de Chung, Garey et Tarjan sur l'orientation d'un graphe mixte [2] met en pratique les résultats de Robbins [7] et Boesch et Tindell [1], qui ont prouvé qu'on pouvait orienter un graphe mixte G de manière à ce qu'il soit fortement connexe si G est connexe et ne contient pas de bridge. En effet, cet article [2] nous donne un algorithme qui nous permet d'orienter un graphe mixte sans pour autant optimiser cette orientation. Cependant, cet algorithme peut servir d'heuristique et de point de départ pour des métaheuristiques.

Nous commençons par poser les définitions nécessaires à la compréhension de l'algorithme.

Voici le pseudo-code d'un DFS simple, cet algorithme est récursif, et prend en argument, un graphe G orienté ou non :

Définition 19. *Un **DFS (Depth-First Search)** (parcours en profondeur) est un algorithme qui explore les sommets d'un graphe en s'enfonçant au maximum avant de revenir en arrière. Le DFS est à la base de nombreux algorithmes, dont celui de Tarjan.*

Algorithm 1: DFS(G)

Result: tree : arborescence avec comme racine v

```

1 seen  $\leftarrow \emptyset$ ;
2 tree  $\leftarrow \emptyset$ ;
3 foreach  $i \in V$  do
4   if  $i \notin \text{seen}$  then
5      $\text{explore}(G, i, \text{seen}, \text{tree})$ ;
6   end
7 end
```

Algorithm 2: $\text{explore}(G, \text{sommet}, \text{seen}, \text{tree})$

```

1 seen  $\leftarrow \text{seen} \cup \text{sommet}$ ;
2 foreach  $i \in \Gamma^+(\text{sommet})$  do
3   if  $i \notin \text{seen}$  then
4     tree  $\leftarrow \text{tree} \cup \{(\text{sommet}, i)\}$ ;
5     seen  $\leftarrow \text{seen} \cup \{i\}$ ;
6      $\text{explore}(G, i, \text{seen}, \text{tree})$ 
7   end
8 end
```

Définition 20. *Une **tree arête** est une arête qui quand elles sont traversées lors du DFS conduisent à des sommets précédemment non atteints.*

Définition 21. *Une **non-tree arête** est une arête qui quand elles sont traversées lors du DFS conduisent à des sommet déjà atteints.*

Définition 22. *Le **Préfixe** est un numéro, lors d'un DFS, attribué à chaque sommet correspondant à son ordre de passage au moment de la première visite du sommet.*

Définition 23. *La fonction **Low**(v) associée à un sommet v est la plus petite valeur de préfixe atteignable depuis v (y compris lui-même), en suivant des tree arête et au plus une unique non-tree arête. Elle est utilisée notamment pour détecter les points d'articulation ou les composantes fortement connexes.[9]*

Dans leur algorithme Chung et al utilisent un DFS : ils prennent celui de Tarjan [9] et le modifient tout en utilisant la fonction $\text{low}(v)$

Dans cette partie on définira $[v, w]$ comme étant un arc ou une arête.

5.1 Quelques lemmes dûs à Chung et al.

Dans leur papier Chung, Garey et Tarjan introduisent 4 lemmes que nous allons présenter ici.

Lemme 1. *Soient G un graphe orientable, c un cycle de G et $\{v, w\}$ une arête de c , alors le graphe mixte formé en orientant $\{v, w\}$ dans la direction consistante avec c est orientable.*

Démonstration. On suppose $\vec{G}'$ est un graphe formé à partir de $\vec{G}$ en orientant toutes les arêtes de c dans la direction consistante à c . Si $[x, y]$ de c , alors il y a un chemin dans $\vec{G}'$ de x à y , et un autre de y à x . On a donc $\vec{G}'$ fortement connexe, car pour tout chemin p de $\vec{G}$ il y a p' dans $\vec{G}'$, formé en substituant le chemin approprié pour chaque arête de p qui est aussi dans c . Comme $\vec{G}'$ est une orientation de G , alors le lemme est vérifié. $\square$

Lemme 2. *Soient G un graphe orientable, $X, \bar{X}$ une partition quelconque de G tel qu'une seule arête e traverse la partition de X à $\bar{X}$ et que tous les arcs traversent la partition dans le sens de X vers $\bar{X}$. Alors, le graphe $\vec{G}'$ formé en orientant e de $\bar{X}$ à X est orientable.*

Démonstration. Comme tous les arcs traversent de X vers $\bar{X}$ si on oriente aussi e dans ce sens alors le graphe résultant ne sera pas fortement connexe car nous ne pourrons plus aller de $\bar{X}$ à X . $\square$

Lemme 3. *Pour tout sommet v , tel que $low(v) \leq v$, et il y a un chemin de v à $low(v)$ dans $\vec{G}$. Si v n'est pas le sommet de départ, alors quand le DFS se retire de v , $low(v) < v$. Donc si l'algorithme ne s'arrête pas en court de route, il y a un chemin dans $\vec{G}$, de tout sommet v à 1, le sommet de départ.*

Démonstration. Soit v le sommet que nous sommes en train d'explorer lors de la remontée et on traite l'arête/arc $[w, v]$ du DFS, on a donc $low(w) < low(v) \leq v$ car w est avant v dans l'arbre. Il y a 3 cas possibles :

1. $[w, v]$ est non-tree arête/arc (voir la définition plus loin) alors $low(v) = low(w) < v$.
2. $low(v) < v$ trivial.
3. $[w, v]$ tree arête/arc et $low(v) = v$ alors $low(v) = low(w) < v$.

Donc on a bien $\forall v \in V, low(v) < v$. Cela signifie que l'on peut aller de v à $low(v)$ et donc de proche en proche on peut aller de tous sommets v au sommet de départ du DFS 1. $\square$

Lemme 4. *Si tous les sommets ne sont pas atteints pendant le DFS, ou si on arrête le DFS alors G n'est pas orientable. Si G est orientable alors $\vec{G}$ est fortement connexe.*

Démonstration. On considère une retraite d'un sommet w à un autre sommet v sachant que $[v, w]$ est une tree-arête.

On suppose que $low(w) < w$ juste avant la retraite si et seulement si il y a $[x, y]$ dans G tel que x et non y soit un descendant de w dans l'arbre généré par le DFS (donc w est un son propre descendant). S'il y a un tel arc/arête alors la nature du DFS implique que $y < w \leq x$.

Après le retrait le long de $[x, y]$ on aura donc le cas où $low(x) \leq y$. Cette valeur va se propager de w à x , pour qu'on ait à la fin $low(w) \leq y < w$. Inversement, si $low(w) < w$, $low(w)$ n'est pas un descendant de w , et le chemin de w à $low(w)$ doit contenir $[x, y]$ tel que x , mais pas y , soit un descendant de w . Supposons que la recherche échoue dans le cas 2 lors d'un recul le long d'une arête (v, w) , c'est-à-dire que $low(w) = w$ juste avant le recul. Soit X les descendants de w et $\bar{X}$ les sommets restants. Alors, d'après l'affirmation ci-dessus, chaque arc traversant la coupure $X, \bar{X}$ est orientée dans G de $\bar{X}$ vers X , et il n'y a pas d'arête qu'on pourrait orienter de X vers $\bar{X}$. Cela signifie donc que G n'est pas orientable, car $\vec{G}$ ne sera pas fortement connexe d'après le lemme 2. $\square$

5.2 Description de l'algorithme

L'algorithme est linéaire, et c'est une variante de l'algorithme de Tarjan permettant de trouver les composantes fortement connexes dans un graphe en utilisant un DFS [9].

Cet algorithme comporte deux parties. La première oriente toutes les arêtes de G en appliquant les lemmes 1 et 2. La seconde vérifie que le graphe orienté résultant est bien fortement connexe.

Pour guider le processus d'orientation, nous maintenons, pour chaque sommet v , un nombre $low(v)$ tel

que $1 \leq low(v) \leq prefixe(v)$. Il y aura toujours dans le graphe actuel une chaîne de v à $low(v)$ par définition. On initialise $\forall v \in V, low(v) = prefixe(v)$.

Dans la première partie on effectue un DFS de G à la manière de Tarjan, décrit dans l'algorithme 8, en commençant par un sommet quelconque que l'on nommera la **racine**. On attribue les numéros préfixes des sommets de 1 à n au fur et à mesure qu'ils sont atteints pendant le DFS. Si tous les sommets ne sont pas atteints alors G n'est pas orientable car il n'est déjà pas connexe.

Pendant le DFS, lors de la remontée, donc lorsqu'on a atteint un sommet duquel on ne peut plus atteindre de nouveau sommet, on commence l'orientation des arêtes. Pendant le processus d'orientation on met à jour la numérotation low et on applique les cas suivant quand on rencontre une arête ou un arc $[v, w]$ reculant de w à v , si c'est une non-tree alors on la considère dans le sens $[w, v]$.

1. On commence par les non-tree, $[w, v]$: on remplace $low(w)$ par $\min\{low(v), low(w)\}$. Dans le cas où $[w, v]$ est une arête, on l'oriente de w vers v . On oriente dans ce sens car si $[w, v]$ est une non-tree arête alors on peut déjà aller de v à w en suivant le sens de l'arbre obtenu à l'aide du DFS.
2. Ensuite, on s'occupe des tree arêtes/arcs $[v, w]$. Il n'y en qu'une par sommet car le DFS forme un arbre. On a deux cas :
 - Si $low(w) < prefixe(w)$ alors on remplace $low(v)$ par $\min\{low(v), low(w)\}$. Dans le cas où $[v, w]$ est une arête, on l'oriente de v vers w .
 - Si $low(w) = prefixe(w)$. Si $[v, w]$ est orientée de v à w alors on arrête l'algorithme : G n'est pas orientable d'après le lemme 2. Si $[v, w]$ n'est pas orientée, on oriente de w à v et on remplace $low(w)$ par $low(v)$.

La première partie de l'algorithme s'exécute en temps $\Theta(n + m)$ sur un graphe mixte de n sommets et m arêtes.

La deuxième partie de l'algorithme consiste à effectuer un DFS depuis la même racine afin de vérifier si, dans G , tous les sommets sont accessible depuis la racine. En effet la première partie de l'algorithme assure par les choix d'orientations de toujours pouvoir remonter le long de l'arbre donné par le DFS. On vérifie maintenant qu'on peut descendre. Après la deuxième partie, si tous les sommets sont atteints à partir de la racine alors le lemme 3 implique que G est fortement connexe et donc orientable. Dans le cas contraire, le lemme 4 implique que G n'est pas orientable.

La deuxième partie s'exécute en temps $\Theta(n + m)$ également. On considère que l'algorithme en entier s'exécute en temps linéaire.

5.3 Pseudocode de Chung, Garey et Tarjan

Algorithm 3: Heuristique(G)

Result: $\vec{E}$: l'orientation des arêtes

- 1 $M \leftarrow$ Matrice d'adjacence(G);
- 2 $seen \leftarrow \emptyset$;
- 3 $sommet \leftarrow$ sommet aléatoire de V ;
- 4 $tree \leftarrow \emptyset$;
- 5 $low \leftarrow$ dictionnaire vide;
- 6 $prefixe \leftarrow$ dictionnaire vide;
- 7 $\vec{E} \leftarrow \emptyset$;
- 8 $DFS_Orientation(M, seen, sommet, tree, low, E, prefixe, \vec{E})$;
- 9 **return** $\vec{E}$;

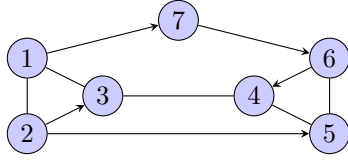
La fonction `DFS_Orientation` est décrite dans l'algorithme 8, en Annexe C, correspond à la première étape de l'algorithme proposé par Chung et al. Nous ne retenons pas la seconde partie de l'algorithme, qui consiste uniquement à vérifier si le graphe orienté obtenu est fortement connexe. En effet, cette vérification est inutile dans notre cas, puisque l'heuristique `Heuristique` n'est appliquée qu'à des graphes orientables, pour lesquels la forte connexité est déjà garantie.

5.4 Exemples concrets

Dans cette section nous allons faire 3 exemples dont 2 avec des graphes qui ne sont pas orientables.

5.4.1 Exemple I : Un graphe orientable

Cet exemple est tiré de l'article de Chung et al [2].

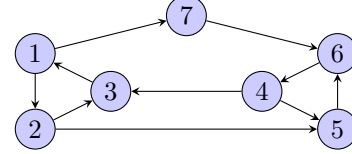


En faisant le DFS on arrive au sommet 6 qui est le premier sommet que l'on va fermer. On commence donc à orienter les arêtes :

- (6, 4), non-tree arête $\implies low(6) = low(4) = 4$
- (5, 6), $low(6) = 4 < 6 \implies low(5) = low(6) = 4$ et $[5, 6]$
- (4, 5), $low(5) = 4 < 5 \implies [4, 5]$

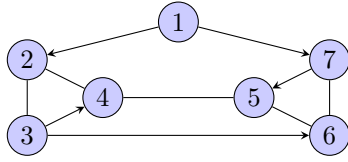
- (3, 4), $low(4) = 4$ et tree arête $\implies low(4) = low(3) = 3$ et $[4, 3]$
 - (3, 1), non-tree arête $\implies low(3) = low(1) = 1$ et $[3, 1]$
 - (2, 3), $low(3) = 1 < 2 \implies low(2) = low(3) = 1$ et $[2, 3]$
 - (2, 5), non-tree arête $\implies \emptyset$
 - (1, 2), $low(2) = 1 < 2 \implies [1, 2]$
- on explore 7 depuis 1 :
- (7, 6), non-tree arête $\implies low(7) = low(6) = 4$
 - (1, 7), $low(7) = 4 < 2 \implies \emptyset$

Le graphe résultant est le suivant :



5.4.2 Exemple II : Un graphe non orientable

Le graphe suivant n'est pas orientable, car $\Gamma^+(1) = \emptyset$ donc il n'est pas connexe.



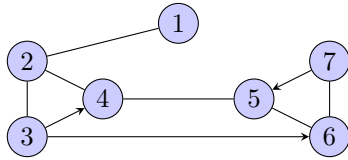
En faisant le DFS on arrive au sommet 7 qui est le premier sommet que l'on va fermer. On commence donc à orienter les arêtes.

- (7, 5), non-tree arête $\implies low(7) = 5$
- (6, 7), $low(7) = 5 < 7 \implies low(6) = low(7) = 5$ et $[6, 7]$
- (5, 6), $low(6) = 5 < 6 \implies [5, 6]$

- (4, 5), $low(5) = 5$ et tree arête $\implies low(5) = low(4) = 4$ et $[5, 4]$
- (4, 2), non-tree $\implies low(4) = low(2) = 2$ et $[4, 2]$
- (3, 4), $low(4) = 4 < 4 \implies low(3) = low(4) = 2$ et $[3, 4]$
- (3, 6) non-tree arête $\implies \emptyset$
- (2, 3), $low(3) = 2 < 3 \implies [2, 3]$
- (1, 2), $low(2) = 2$ et tree arête $\implies$ abandon car $[1, 2]$ est orientée.

5.4.3 Exemple III : Encore un graphe non orientable

Le graphe suivant n'est pas orientable, car $\{1, 2\}$ est un bridge.

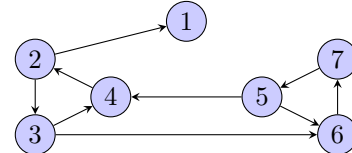


En faisant le DFS on arrive au sommet 7 qui est le premier sommet que l'on va fermer. On commence donc à orienter les arêtes.

- (7, 5), non-tree arête $\implies low(7) = 5$
- (6, 7), $low(7) = 5 < 7 \implies low(6) = low(7) = 5$ et $[6, 7]$
- (5, 6), $low(6) = 5 < 6 \implies [5, 6]$

- (4, 5), $low(5) = 5$ et tree arête $\implies low(5) = low(4) = 4$ et $[5, 4]$
- (4, 2), non-tree $\implies low(4) = low(2) = 2$ et $[4, 2]$
- (3, 4), $low(4) = 4 < 4 \implies low(3) = low(4) = 2$ et $[3, 4]$
- (3, 6) non-tree arête $\implies \emptyset$
- (2, 3), $low(3) = 2 < 3 \implies [2, 3]$
- (1, 2), $low(2) = 2$ et tree arête $\implies low(2) = low(1)$ et $[2, 1]$

Voici le graphe résultant :



Cet exemple illustre l'importance du DFS en fin d'algorithme. Le graphe résultant après la première étape n'est pas fortement connexe, car on ne peut pas aller de 1 vers 2 par exemple.

6 Application à notre problème

Pour appliquer l’algorithme 3 il faut un graphe sans bridge or ce n’est pas notre cas, et nous pouvons toujours décider de laisser les bridges en double sens. Du moment que G est connexe, il est orientable quitte à faire ce choix. Donc avant d’appliquer l’algorithme 3, nous faisons quelques modifications sur G .

6.1 Rendre G orientable

Premièrement, nous allons faire en sorte que notre graphe n’ait pas de bridge, car l’heuristique ne fonctionne que sur des graphes sans bridge. Pour cela, dans l’algorithme 4 nous allons identifier les composantes 2-connexes qui sont fortement connexes de notre graphe G . Donc à l’aide de la fonction de networkx (python) `nx.bridges(G)` on identifie les bridges et on les enlève de G et on note se nouveau graphe $G_{bridgeless}$.

On note aussi $G_{bridgeless}^1 \dots G_{bridgeless}^k$ les composantes fortement connexes de $G_{bridgeless}$ qui sont identifiées à l’aide de la fonction de networkx `nx.strongly_connected_components(G)`. On applique donc l’algorithme 3 sur les $G_{bridgeless}^i$ pour les orienter. On a donc $\vec{G} = \cup_{i \in [1,k]} \vec{G}_{bridgeless}^i \cup Bridges$. $Bridges$ correspond au bridges de G qui ont été enlevés au début.

Enfin, on veut pouvoir générer de multiples solutions pour pouvoir les utiliser dans les métaheuristiques. Pour cela, on ajoute de l’aléatoire dans le choix de la racine et de l’ordre de parcours des voisins lors du DFS.

Algorithm 4: RendOrientable(G')

Input: $G = (V, E, U)$

Result: $\vec{G}$: Orientation de G

```

1  $\vec{E} \leftarrow \emptyset$ ;
2  $sccs \leftarrow \text{subGraph}(G)$ ;
3 foreach  $X \in sccs$  do
4    $\vec{E} \leftarrow \vec{E} \cup \text{Heuristique}(X)$ ;
5 end
6  $\vec{G} \leftarrow (V, U \cup \vec{E})$ ;
7 return  $\vec{G}$ ;
```

La fonction `subGraph( $G$ )` fait une copie de G dans $G_{bridgeless}$. Ensuite, elle supprime les bridges de $G_{bridgeless}$ et enfin, renvoie les composantes fortement connexes de $G_{bridgeless}$.

6.2 Prétraitement

Dans cette partie nous allons parler de pré-traitement. Donc des modifications que l’on peut faire sur G pour diminuer le temps de calcul de nos algorithmes.

En effet, avant d’appliquer un algorithme à G on peut diminuer le nombre de contraintes en vérifiant s’il y a des arêtes dont le sens d’orientation est fixé d’après le lemme 2 de Chung et al [2]. De même, dans le choix des couples (s,t) on peut enlever ceux qui ne passent pas par des arêtes car leur plus court chemin ne sera jamais impacté par l’orientation des arêtes.

6.2.1 Orientation des strong bridges

Les arêtes qui ont une orientation fixée d’après le lemme 2 de Chung et al [2] correspondent à ce qu’on appelle des strong bridges (voir définition plus haut). Dans notre cas lorsqu’on enlève un strong bridge de notre graphe celui-ci n’est plus fortement connexe.

Trouver les strong bridges permet de diminuer la taille de E et donc le nombre d’orientations possible de G , car une seule orientation est possible pour ces strong bridges.

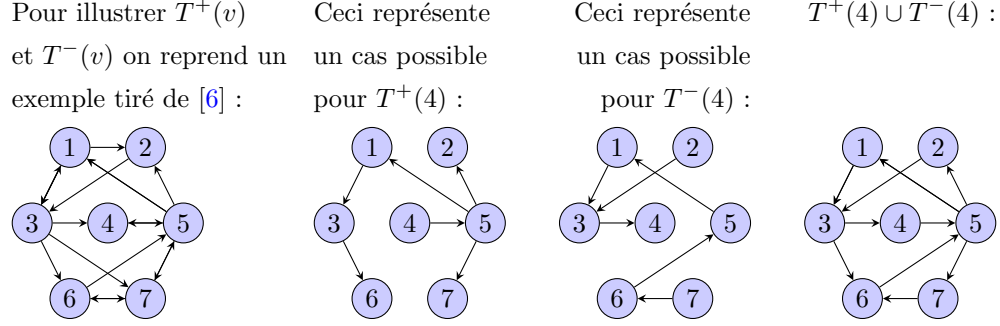
Pour trouver tous les strong bridges on va utiliser l’article de recherche de Giuseppe F. Italiano et al [6].

Plusieurs choses sont définies dans cet article mais ce qui nous intéresse ici c’est : pour v un sommet quelconque de G on note :

Définition 24. $T^+(v)$ désigne l’arbre couvrant orienté obtenu en effectuant un parcours en profondeur (DFS) du graphe G , en partant du sommet v comme racine. Il représente un arbre dirigé allant de v vers tous les autres sommets accessibles depuis v .

Définition 25. $T^-(v)$ est construit à partir du graphe inverse G^{-1} . On effectue un DFS depuis le sommet v , considéré comme racine dans G^{-1} , puis on inverse l’orientation de toutes les arêtes de

l'arbre obtenu. Le résultat est un arbre dirigé dont v est anti-racine. On rappelle qu'une anti-racine est un sommet atteignable par tous les autres sommets.



On a les propriétés suivantes :

Propriété 1. $\forall v \in V$, $T^+(v)$ et $T^-(v)$ contiennent $(n - 1)$ arcs dans le cas d'un graphe fortement connexe.

Propriété 2. $\forall v \in V$, $T^+(v) \cup T^-(v)$ contient au plus $(2n - 2)$ arcs (car il peut y avoir des arcs appartenant à $T^+(v)$ et $T^-(v)$ en même temps).

De plus on a le théorème suivant :

Théorème 3. $\forall v \in V$, $G' = T^+(v) \cup T^-(v)$ contient tous les strong bridges de G .

Démonstration. En effet, pour toute paire de sommets x, y de V il y a un chemin de x vers y dans G' : $T^-(v)$ contient un chemin de x à v et $T^+(v)$ contient un chemin de v à y donc $T^+(v) \cup T^-(v)$ contient un chemin de x à y donc G' est un sous-graphe de G fortement connexe. Or tous sous-graphes de G fortement connexe contiennent tous ses strong bridges par définition. $\square$

Donc une fois $T^+(v) \cup T^-(v)$ obtenue on peut vérifier pour chaque arc si sa suppression entraîne la perte de forte connexité du graphe. Pour obtenir $T^+(v)$ on fait un DFS en commençant avec v et pour obtenir $T^-(v)$ on fait pareil un DFS à partir de v mais on fait le DFS de G inverse -c'est à dire $G^{-1} = (V, A'^{-1})$ avec $A'^{-1} = \{(u, v) / (v, u) \in A'\}$.

Algorithm 5: StrongBridges(G)

Result: ensemble des strong bridges

```

1  $v \leftarrow$  un sommet aléatoire de  $G$ ;
2  $G_{inverse} \leftarrow G^{-1}$ ;
3  $seen \leftarrow \emptyset$ ;
4  $T^+(v) \leftarrow \emptyset$ ;
5  $T^-(v) \leftarrow \emptyset$ ;
6 DFS( $G, v, seen, T^+(v)$ );
7  $seen \leftarrow \emptyset$ ;
8 DFS( $G_{inverse}, v, seen, T^-(v)$ );
9  $T^-(v) \leftarrow \{(i, j) \text{ foreach } (j, i) \in T^-(v)\}$ ;
10  $G' \leftarrow T^+(v) \cup T^-(v)$ ;
11  $strong\_bridges \leftarrow \emptyset$ ;
12 foreach  $e \in G'$  do
13   if  $e \in E$  then
14      $G \leftarrow G \setminus e$ ;
15     if  $G$  pas fortement connexe then
16        $strong\_bridges \leftarrow strong\_bridges \cup \{e\}$ ;
17     end
18      $G \leftarrow G \cup e$ ;
19   end
20 end
21 return  $strong\_bridges$ ;

```

Ainsi, la fonction **StrongBridges**(G), l'algorithme 5, permet d'identifier, dans le graphe mixte G , les arêtes dont l'une des deux orientations possibles constitue un strong bridge. Pour calculer $T^+(v)$ et $T^-(v)$ on utilise la fonction DFS correspondant à l'algorithme 1. Pour garantir la forte connexité du graphe une fois toutes les arêtes orientées, ces arêtes doivent obligatoirement être orientées dans le sens du strong bridge détecté. Cette fonction permet donc de déterminer l'ensemble des arêtes dont l'orientation est contrainte par la structure du graphe.

6.2.2 Choix stratégique des couples (s,t)

Après avoir orienté les strong bridges, on choisit les couples (s,t) qui sont pertinents. C'est à dire ceux dont leur plus court chemin passe par au moins une arête -qui n'est pas un strong bridge- et donc dont l'orientation de G impactera leur plus court chemin. Pour cela on a implémenté l'algorithme 6.

Algorithm 6: `Candidat_st(G)`

Result: $\vec{G}$: Orientation fortement connexe de G

```
1 path  $\leftarrow$  all_pairs_dijkstra_path( $G$ );
2 candidat  $\leftarrow$  liste vide;
3 foreach  $i \in V$  do
4   foreach  $j \in V$  do
5     if  $i \neq j$  then
6       p  $\leftarrow$  path[i][j];
7       foreach  $k \in \llbracket 0, \text{len}(p) - 1 \rrbracket$  do
8         if  $\{p[k], p[k + 1]\} \in E$  then
9           candidat  $\leftarrow$  candidat + (i, j);
10          break;
11        end
12      end
13    end
14  end
15 end
16 return candidat;
```

La fonction `all_pairs_dijkstra_path(G)` est une fonction de Networkx (python) qui renvoie un dictionnaire de dictionnaire contenant les plus petits chemins entre chaque paires de sommets de G .

Comme l'heuristique n'optimise pas la solution cela ne répond qu'à une partie de notre problème. C'est à ce moment que la méta-heuristique rentre en jeux. Cette partie a été réalisé par Léon Hogenmuller, je ne la développerai donc pas. Mais pour résumer, à partir d'une solution donnée par l'heuristique Léon a mis en place un algorithme que l'on appelle méta-heuristique qui va améliorer cette solution de base.

7 Méta-heuristique

La métaheuristique que nous avons choisie est la Recherche Tabou, aussi appelée *Tabu Search* en anglais. Il s'agit d'une méthode d'optimisation introduite par Glover en 1989 [5]. C'est un algorithme itératif qui explore l'espace des solutions à partir d'une solution initiale. À chaque itération, l'algorithme évalue le voisinage de la solution courante et se déplace vers la meilleure solution voisine, même si celle-ci ne constitue pas une amélioration directe.

La particularité de la *Tabu Search*, par rapport à un algorithme de descente classique, réside dans la possibilité pour l'algorithme de dégrader ponctuellement la solution. Dans ce type d'algorithme, nous utilisons une liste tabou. Celle-ci mémorise certains mouvements ou solutions récemment explorés et interdit temporairement de les revisiter, empêchant ainsi l'algorithme de retomber trop rapidement dans les mêmes configurations. Cette stratégie permet de sortir des minima locaux et d'encourager une exploration plus diversifiée de l'espace de recherche.

Bien que cette approche puisse occasionnellement entraîner une dégradation temporaire de la fonction objectif, elle offre à long terme la possibilité de découvrir des solutions de meilleure qualité, tout en évitant les cycles (enchaînement de visites de solutions dans le même ordre, en boucle).

Algorithm 7: Tabu Search

Input: s_0 : solution initiale
Input: it : nombre maximum d'itérations
Input: $maxTabuSize$: taille maximale de la liste tabou
Input: $fitness$: fonction d'évaluation
Input: $getNeighbors$: fonction générant les voisins
Input: A, E : paramètres du problème

```
1  $s_{Best} \leftarrow s_0$ ;  
2  $s_{Curr} \leftarrow s_0$ ;  
3  $bestCandidate \leftarrow s_0$ ;  
4  $tabuList \leftarrow \{s_0\}$ ;  
5  $n \leftarrow 0$ ;  
6 while  $n < it$  do  
7    $s_{Neighborhood} \leftarrow getNeighbors(s_{Curr}, A, E)$ ;  
8    $bestCandidateFitness \leftarrow +\infty$ ;  
9   foreach  $s_{Candidate} \in s_{Neighborhood}$  do  
10    if  $s_{Candidate} \notin tabuList$  and  $fitness(s_{Candidate}, A, E) < bestCandidateFitness$  then  
11       $bestCandidate, bestCandidateMovement \leftarrow s_{Candidate}$ ;  
12       $bestCandidateFitness \leftarrow fitness(s_{Candidate}, A, E)$ ;  
13   if  $bestCandidateFitness = +\infty$  then  
14     break;  
15    $s_{Curr} \leftarrow bestCandidate$ ;  
16    $tabuList \leftarrow tabuList \cup bestCandidateMovement$ ;  
17   if  $|tabuList| > maxTabuSize$  then  
18      $tabuList.pop()$ ;  
19   if  $fitness(s_{Best}, A, E) > bestCandidateFitness$  then  
20      $s_{Best} \leftarrow bestCandidate$ ;  
21    $n \leftarrow n + 1$ ;  
22 return  $s_{Best}$ 
```

On peut voir ci-dessus le pseudocode de notre algorithme de *Tabu Search*, inspiré de celui présenté sur Wikipédia [8].

Les fonctions **fitness** et **getNeighbors** jouent un rôle central dans notre implémentation, car elles déterminent à la fois l'évaluation des solutions et la manière dont l'espace de recherche est exploré. Dans ce travail, nous avons porté une attention particulière à leur conception, afin de respecter les contraintes du problème tout en optimisant l'objectif fixé.

Une première approche pour définir la fonction **fitness** a consisté à utiliser la moyenne des plus courts chemins dans le graphe, mesurant ainsi l'efficacité globale de la structure. Cependant, cette méthode présente une limite importante : son coût de calcul élevé. En effet, le calcul de tous les plus courts chemins est en $O(n^3)$ (par exemple via l'algorithme de Floyd-Warshall), ce qui, répété à chaque itération, rend l'algorithme particulièrement lent pour des graphes de taille moyenne à grande. Nous avons donc pensé à une nouvelle approche pour cette fonction **fitness** : ne plus recalculer les plus courts chemins entre chaque paire de sommets du graphe, mais plutôt essayer de recalculer dynamiquement, à chaque itération, les plus courts chemins des couples impactés par le changement. Cette méthode nous permet également de ne pas calculer les plus courts chemins de tous les couples (s, t) du graphe : comme expliqué dans le prétraitement, certains plus courts chemins ne passent pas par des routes de E , ils ne seront donc jamais modifiés. Il n'est donc pas nécessaire de s'y intéresser.

Concernant la fonction **getNeighbors**, nous avons choisi de générer l'ensemble des graphes fortement connexes obtenus en modifiant l'orientation d'une arête par rapport à la solution courante. Cette approche garantit que tous les voisins explorés respectent la contrainte de forte connexité, tout en permettant une exploration riche et structurée de l'espace des solutions. Cependant, après quelques tests, nous avons remarqué que ce voisinage était limité en termes de quantité d'individus. Nous avons

donc défini une nouvelle structure de voisinage, constituée des graphes ayant une ou deux modifications d'orientation. Cette approche nous a menés à de bien meilleurs résultats.

Enfin, notre liste tabou, contrairement à certaines définitions du *Tabu Search*, ne contient pas les solutions précédemment visitées, mais les mouvements précédemment choisis. Ainsi, la liste est constituée d'arêtes ou de couples d'arêtes (en fonction des mouvements effectués par l'algorithme) et empêche tout retour en arrière sur ces arêtes. Ce choix limite les risques pour notre algorithme de tomber dans des cycles. Une approche que l'on peut envisager pour améliorer l'efficacité de cet algorithme est le parallélisme du calcul des valeurs de fitness des voisins. En effet, ce calcul n'est pas très coûteux en mémoire, mais le grand nombre de voisins rend le temps d'exécution assez long. Le parallélisme permettrait alors de calculer simultanément la fitness de nombreux voisins afin de les comparer par la suite.

8 Résultats

Plusieurs expérimentations numériques ont été menées afin d'évaluer les performances des méthodes proposées. L'ensemble des calculs a été réalisé sur un ordinateur fixe équipé d'un processeur AMD Ryzen 5 9600X (fréquence maximale de 5,4 GHz), avec 32 Go de mémoire vive, fonctionnant sous Windows 11 Professionnel. Le code a été développé en Python et exécuté en mode mono-thread (single-threaded). Aucune forme de parallélisation n'a été utilisée, ce qui permet une évaluation plus neutre des performances algorithmiques. L'environnement de développement utilisé est Visual Studio Code, avec les bibliothèques standards de calcul scientifique en Python, notamment **NetworkX** et **NumPy**.

8.1 Génération des données

Comme cette version du problème de forte orientation n'a pas encore été largement étudiée, il n'existe pas d'instances de test standardisées dans la littérature. Il nous a donc été nécessaire de générer manuellement les instances utilisées dans ce travail. Afin que nos résultats reflètent au mieux les caractéristiques du monde réel, nous avons choisi de nous appuyer sur des données géographiques authentiques.

Pour cela, nous avons utilisé le package Python **OSMnx**, qui s'appuie sur les données d'OpenStreetMap. Cet outil permet d'extraire automatiquement la structure du réseau viaire d'un quartier, d'une ville ou d'une zone géographique donnée, et de la représenter sous forme de graphe orienté ou non orienté. Je ne détaillerai pas cette partie, car elle a été réalisée par Léon Hogenmuller. Il constitue ainsi une base idéale pour générer des graphes réalistes à partir de configurations urbaines existantes.

Les graphes ainsi obtenus sont directement compatibles avec le package **NetworkX**, également utilisé dans nos algorithmes. Toutefois, ces graphes extraits sont des sous-graphes du réseau viaire global et ne peuvent pas être exploités tels quels pour notre problématique. Deux contraintes essentielles motivent un prétraitement : le graphe doit être fortement connexe et ne doit contenir aucun bridge.

Nous avons donc mis en place une procédure d'extraction permettant d'isoler les sous-graphes satisfaisant ces contraintes.

Dans un premier temps, nous utilisons la fonction `graph_from_place` d'**OSMnx** pour récupérer le graphe d'une zone géographique donnée. Ce graphe est ensuite converti en un graphe simple à l'aide de **NetworkX**. Nous en extrayons ensuite la composante fortement connexe principale, en utilisant la fonction `strongly_connected_components` de **NetworkX**. Les autres composantes, généralement très petites (souvent moins de cinq nœuds), sont négligées car elles correspondent à des segments périphériques sans intérêt pour notre étude.

Enfin, concernant les ponts (bridges), leur présence ne remet pas en cause notre capacité à orienter le graphe. En effet, tant que ces arêtes sont conservées à double sens, la propriété de forte connexité reste assurée. Nous choisissons donc de ne pas les supprimer, mais simplement de les traiter comme des arêtes à double sens fixes, ce qui est compatible avec notre modèle d'orientation.

8.2 Tests sur le Prétraitement

Concernant les tests je ne me suis occupé que des tests concernant le prétraitement et Léon s’est occupé des autres tests.

La table 3 présente les données de base pour chaque instance, qui serviront de référence. On y trouve notamment la valeur de λ_{total} , correspondant à la somme des distances des plus courts chemins entre

toutes les paires de sommets distincts de G avant toutes modifications :

$$\lambda_{total} = \sum_{\substack{s,t \in V \times V \\ s \neq t}} \lambda_{st}$$

On observe que, globalement, les instances de plus petite taille comptent davantage de strong bridges, avec quelques exceptions. Par ailleurs, l’identification des paires OD pertinentes permet de réduire significativement le nombre total P_{total} dans les instances ayant peu d’arêtes et/ou un grand nombre de strong bridges. Cependant, cette réduction n’est pas systématique pour toutes les instances.

La table 4 compare les temps de résolution du PLNE avec et sans prétraitement, mettant en évidence un gain de temps important grâce à ce dernier. Dans cette table, z^* représente la valeur optimale de la solution. Le pourcentage de dégradation de la distance totale des plus courts chemins dans $\tilde{G}$ par rapport à G est calculé comme suit : $\frac{z^* - \lambda_{total}}{\lambda_{total}} \times 100$. Enfin, le gain de temps obtenu avec le prétraitement est quantifié par : $\frac{tps_sans - tps_avec}{tps_sans} \times 100$.

On remarque que le pourcentage de dégradation reste inférieur à 30 % pour la majorité des instances. De plus, le gain de temps est conséquent, près de la moitié des instances bénéficiant d’un gain supérieur à 50 %.

Enfin, comme le PLNE devient coûteux en temps de calcul en raison du nombre élevé de variables, réduire la taille de P permet de diminuer le temps nécessaire à sa résolution. Cependant, lorsqu’on ne considère pas l’ensemble complet $P_{candidat}$, il n’existe plus de garantie que la solution soit fortement connexe ni qu’elle corresponde à la solution optimale. La table 5 présente donc l’impact de la réduction du nombre de paires OD sur le temps de calcul et la qualité de la solution du PLNE. Pour les instances qui produisent des graphes fortement connexes quelle que soit la taille de P , seules les données correspondant aux tailles maximale et minimale de P sont indiquées.

On observe que, pour de nombreuses instances, l’écart entre la valeur obtenue de la fonction objectif et la valeur optimale varie peu, quel que soit le pourcentage de $P_{candidat}$ retenu. Toutefois, ce pourcentage diffère fortement d’une instance à l’autre, pouvant aller de 80 % à 5 %. Par ailleurs, certaines solutions produisent des graphes qui ne sont pas fortement connexes.

Instance	V	A	E	P_{total}	Strong bridges	$P_{candidat}$	λ_{total}
Saint-Louis, Vincennes	32	50	6	992	6	0	552 788,33
Vignerons, Vincennes	28	51	12	756	11	203	409 732,8113
Domaine du bois, Vincennes	33	60	9	1056	1	889	495 558,8403
République, Vincennes	33	62	15	1056	4	920	544 752,3728
Centre-ville, Vincennes	40	72	13	1560	4	1303	859 182,4044
Centre ville, Montreuil	25	44	10	600	8	193	359 937,8001
Sorano, Vincennes	45	100	34	1980	10	1769	1 047 840,979
Diderot, Vincennes	40	83	28	1560	3	1478	873 137,2455
Bobillot, Montreuil	84	148	16	6972	8	3462	5 773 473,621
Ruffins, Montreuil	69	136	37	4692	15	3933	3 414 931,089
Yvoire, FRANCE	50	97	21	2450	1	2241	1 843 510,349
Barbusse, Montreuil	84	68	44	6972	12	5548	5 933 227,84

TABLE 3 – Données par Instance

La table 6 donne un aperçu des performances du *Tabu Search*. Elle affiche, pour chaque instance, la valeur optimale z^* obtenue par une méthode exacte, ainsi que les valeurs minimale (z_{min}), maximale (z_{max}) et moyenne ($\bar{z}$) des solutions obtenues sur 10 lancements de l’algorithme. L’écart relatif minimum à λ_{total} ($\Delta\%$), le temps d’exécution moyen (RT en secondes) et le nombre moyen d’itérations ($\bar{k}$) sont également reportés.

Les tests présentés dans le tableau ci-dessus ont été réalisés en effectuant, pour chaque instance, dix lancements de notre *Tabu Search*, chacun avec une graine aléatoire différente. Cela permet de générer

Instance	z^*	Pourcentage de dégradation	Temps avec prétrait.	Temps sans prétrait.	Gain de temps (%)
Saint-Louis, Vincennes	573 044,61	3,664	0	0,72	100
Vignerons, Vincennes	465 250,6211	13,550	0,2612	1,7109	84,73
Domaine du bois, Vincennes	612 586,1741	23,615	5,1023	7,6558	33,35
République, Vincennes	709 060,2175	30,162	40,8162	66,7374	38,84
Centre-ville, Vincennes	1 073 233,969	24,913	79,7101	116,2057	31,41
Centre ville, Montreuil	432 331,5883	20,113	0,2064	0,5806	64,45
Sorano, Vincennes	1 503 867,904	43,521	1 068,3265	1 771,2857	39,69
Diderot, Vincennes	1 126 513,002	29	568,1753	579,1956	2
Bobillot, Montreuil	6 110 397,99	6	952,7756	5 582,4726	83
Ruffins, Montreuil	5 055 581,95	48	2 667,6800	5 418,9610	51
Yvoire, FRANCE	3 103 751,809	68	2 270,0953	2 508,1354	9

TABLE 4 – Comparaison des performances avec et sans prétraitement

des solutions initiales variées et de mieux évaluer la robustesse de la métaheuristique. Les paramètres choisis sont une limite maximale de 100 itérations et une taille maximale de la liste tabou fixée à 7.

D’après les résultats obtenus, on constate que, pour les instances dont la solution optimale z^* peut être calculée avec notre PLNE, cette solution optimale est retrouvée au moins une fois parmi les dix exécutions de notre métaheuristique. Le temps d’exécution est par ailleurs considérablement réduit par rapport à la méthode exacte. Par exemple, pour l’instance Ruffins, le temps de calcul passe de 5419 secondes avec la PLNE à seulement 3,23 secondes avec notre approche. Cette performance est notamment accentuée par un nombre moyen d’itérations relativement faible observé sur nos instances, variant entre 1 et 60, alors même que le nombre maximal d’itérations autorisé était fixé à 100. On trouve donc notre solution optimale très rapidement pour des instances de taille modérée.

Toutefois, lorsque l’on considère des instances de plus grande taille, comme Barbusse, pour lesquelles la méthode exacte ne parvient pas à fournir une solution en un temps raisonnable, on observe une augmentation significative du temps de calcul ainsi que du nombre d’itérations. Cette complexité accrue s’explique par le nombre bien plus important de solutions candidates à explorer, ainsi que par le coût élevé de l’évaluation de la fonction objectif, dû à la taille de l’instance.

Enfin, sur une instance encore plus volumineuse, Houille, nous avons constaté que notre métaheuristique, dans son état actuel de développement, ne parvient pas à produire de solution, même approximative. Cela met en évidence les limites de notre approche pour les très grandes instances et suggère la nécessité d’optimisations supplémentaires ou d’autres méthodes.

En ce qui concerne la qualité moyenne des solutions obtenues, les résultats sont globalement satisfaisants. La moyenne des valeurs obtenues ne dépasse jamais 6 % de l’optimal z^* , avec un écart nul pour certaines instances. Cela démontre que la recherche tabou fournit des approximations de bonne qualité pour des instances de taille raisonnable.

Intéressons-nous maintenant à la dégradation des solutions par rapport à λ_{total} (correspondant à la somme des plus courts chemins de l’instance avant orientation). Le pourcentage de dégradation est présenté dans la colonne $\Delta\%$ du tableau 6. On observe que cette dégradation varie fortement selon les instances, allant de 14 % pour le quartier des Vignerons à 68 % pour la ville d’Yvoire, avec une moyenne de 28 % sur nos instances.

Une telle disparité rend difficile l’établissement d’une tendance générale. Toutefois, ces résultats suggèrent que la structure urbaine propre à chaque instance influence significativement cette dégradation. Par exemple, Yvoire, village montagnard au réseau viaire plus contraint, présente probablement une structure moins dense et moins interconnectée, offrant peu d’alternatives de trajets entre deux points. À l’inverse, des quartiers situés en région parisienne disposent souvent de réseaux plus maillés et redondants, ce qui peut expliquer une augmentation moindre de la dégradation.

Instance	P	% paires OD	tps PLNE(s)	z	% dégradation avec z^*	fortement connexe
Vignerons, Vincennes	150	73,89	0,127	465250,6211	13,55	VRAI
Vignerons, Vincennes	10	4,93	0,043	465250,6211	13,55	VRAI
Domaine du bois, Vincennes	600	66,74	2,94	612586,17	23,62	VRAI
Domaine du bois, Vincennes	10	1,12	0,06	628314,84	26,79	VRAI
République, Vincennes	600	65,65	8,29	709060,22	30,16	VRAI
République, Vincennes	25	2,73	0,11	713073,94	30,90	VRAI
Centre-ville, Vincennes	1000	77,22	16,61	1073233,97	24,91	VRAI
Centre-ville, Vincennes	600	46,33	5,65	1073233,97	24,91	VRAI
Centre-ville, Vincennes	100	7,67	0,23	1073233,97	24,91	VRAI
Centre-ville, Vincennes	25	2,00	0,05	1073233,97	24,91	VRAI
Centre-ville, Vincennes	10	0,80	0,04	—	—	FAUX
Centre ville, Montreuil	150	77,72	0,10	432331,59	20,11	VRAI
Centre ville, Montreuil	10	8,13	0,03	432331,59	20,11	VRAI
Sorano, Vincennes	1400	79,14	218,03	1503867,90	43,52	VRAI
Sorano, Vincennes	800	45,17	58,69	1503867,90	43,52	VRAI
Sorano, Vincennes	100	5,74	0,63	1585982,85	51,36	VRAI
Sorano, Vincennes	50	2,79	0,23	1503867,90	43,52	VRAI
Sorano, Vincennes	25	1,42	0,10	—	—	FAUX
Sorano, Vincennes	10	0,57	0,05	1509810,92	44,09	VRAI
Diderot, Vincennes	1200	81.30081301	157.3765597	1126513.002	29.01900612	VRAI
Diderot, Vincennes	10	0.676589986	0.045200109	1326825.94	51.96075297	VRAI
Bobillot, Montreuil	3000	86,08321377	875,0778973	6110397,99	5,835730655	VRAI
Bobillot, Montreuil	10	0,285795942	0,054968119	6165143,346	6,783952815	VRAI
Ruffins, Montreuil	3000	74,81296758	1497,539795	5056703,418	48,07629451	VRAI
Ruffins, Montreuil	2000	50,40322581	307,0327454	5060093,092	48,17555488	VRAI
Ruffins, Montreuil	200	4,97883993	1,534067631	5070235,132	48,47254595	VRAI
Ruffins, Montreuil	100	2,491280518	0,52859664	5069824,754	48,4605288	VRAI
Ruffins, Montreuil	50	1,244709983	0,734639883	—	—	FAUX
Yvoire, FRANCE	1700	76,64562669	947,6355891	3103751,809	68,36096468	VRAI
Yvoire, FRANCE	1200	53,54752343	349,4026349	3103751,809	68,36096468	VRAI
Yvoire, FRANCE	100	4,490345757	0,732966661	3121370,394	69,31667315	VRAI
Yvoire, FRANCE	50	2,254283138	0,228917837	3265026,455	77,10920119	VRAI
Yvoire, FRANCE	25	1,115573405	0,097040653	—	—	FAUX

TABLE 5 – Comparaison des performances avec moins de paires OD

9 Bilan technique et travaux futurs

Afin de rendre les rues en sens unique sans perdre la forte connexité de nos villes, nous avons développé un PLNE capable de fournir une solution optimale au problème étudié. Toutefois, ce modèle devient rapidement difficile à résoudre même pour des villes de taille moyenne, voire certains quartiers un peu volumineux, en raison de son coût computationnel élevé. Afin de limiter cette complexité, nous avons proposé un mécanisme de prétraitement visant à réduire la taille du problème. On identifie les strong bridges avec un algorithme celui-ci pourrait être encore amélioré, en utilisant à la place un algorithme en temps linéaire. Ensuite on identifie les couples s, t pertinents (dont le plus court chemin passe par au moins une arête). Par ailleurs, au lieu de sélectionner aléatoirement les couples (s, t) dans P , une stratégie plus pertinente consisterait à garantir que chaque sommet apparaisse au moins une fois comme origine ou destination, assurant ainsi une meilleure couverture du graphe.

D'un autre côté, afin de compenser le coût élevé associé à une solution exacte, nous avons exploré une approche alternative : celle d'un algorithme méta-heuristique, dont Léon Hogenmuller, l'autre stagiaire, s'est occupé. Nous avons choisi le Tabu Search, qui nous a semblé bien adapté à la nature de notre problème. Cette méthode s'est révélée efficace, avec des résultats intéressants : le temps d'exécution est nettement inférieur à celui de la méthode exacte, tout en parvenant souvent à retrouver la solution optimale après seulement quelques lancements.

Instance	λ_{total}	z^*	$z_{\min}$	$z_{\max}$	$\bar{z}$	$\Delta\%$	$\bar{RT}$ (s)	$\bar{k}$
Vignerons, Vincennes	409 732,81	465 250,62	465 250,62	465 250,62	465 250,62	14	0,00	1
Domaine du bois, Vincennes	495 558,84	612 586,17	612 586,17	618 946,61	617 038,48	24	0,01	1,90
République, Vincennes	544 752,37	709 060,22	709 060,22	799 184,24	753 480,39	30	0,02	1,90
Centre-ville, Vincennes	859 182,40	1 073 233,97	1 073 233,97	1 086 992,22	1 080 113,10	25	0,03	2,90
Centre-ville, Montreuil	359 937,80	432 331,59	432 331,59	432 331,59	432 331,59	20	0,00	1,70
Diderot, Vincennes	873 137,25	1 126 513,00	1 126 513,00	1 239 286,92	1 193 152,66	29	0,40	10,20
Bobillot, Montreuil	5 773 473,62	6 110 397,99	6 110 397,99	6 110 397,99	6 110 397,99	06	0,16	5,10
Ruffins, Montreuil	3 414 931,09	5 055 581,95	5 055 581,95	5 411 138,71	5 091 137,63	48	3,23	58,20
Yvoire, FRANCE	1 843 510,35	3 103 751,81	3 103 751,81	3 403 874,77	3 237 248,17	68	0,20	5
Barbusse, Montreuil	5 933 227,84	—	7 000 418,93	7 037 905,33	7 022 910,77	18	39,04	100

TABLE 6 – Résultats du Test Tabu Search

Enfin, il serait pertinent d’explorer d’autres fonctions objectifs, qui pourraient mettre en lumière des priorités différentes. En effet, les besoin et choix des politiques ou des individus peuvent différer avec notre approche c’est pourquoi d’autres approches notamment par le biais d’une autre fonction objectif seraient pertinentes. Par exemple, la minimisation du diamètre du graphe pourrait conduire à des solutions mieux adaptées à certains contextes. Ce changement d’approche permettrait de mieux répondre à des besoins variés, qu’ils soient d’ordre politique, social ou opérationnel. Une illustration d’une fonction objectif alternative est proposée en Annexe D (45).

Références

- [1] Frank BOESCH et Ralph TINDELL. “Robbins’ Theorem for mixed multigraphs”. In : *The American Mathematical Monthly* 87.9 (oct. 1980), p. 716-719. URL : <https://www.jstor.org/stable/2303897>.
- [2] Fan R. K. CHUNG, Michael R. GAREY et Robert E. TARJAN. “Strongly Connected Orientation of Mixed Multigraphs”. In : *Networks* 15.4 (1985), p. 477-484. URL : <https://mathweb.ucsd.edu/~fan/myraps/fanpap/79scoomm.PDF>.
- [3] Václav CHVÁTAL et C. THOMASSEN. “Distance in Orientations of Graphs”. In : *Journal of Combinatorial Theory Serie B*.24 (1978), p. 61-75.
- [4] Alessio CONTE et al. “Directing Road Networks by Listing Strong Orientations”. In : (2016). URL : <https://hal.science/hal-01388476v1>.
- [5] Fred GLOVER. “Tabu Search - Part 1”. In : *Journal on Computing* 1 (3) (1989), p. 190-206.
- [6] Giuseppe F. ITALIANO, Luigi LAURA et Federico SANTARONI. “Finding strong bridges and strong articulation points in linear time”. In : *Theoretical Computer Science* 447 (2012), p. 74-84.
- [7] Herbert ROBBINS. “Theorem on graphs, with an application to a problem of traffic controle”. In : *The American Mathematical Monthly* 46.5 (mai 1939), p. 281-283. URL : <https://www.jstor.org/stable/2321858>.
- [8] “Tabu search”. In : (). URL : [https://en.wikipedia.org/wiki/Tabu_search#:~:text=be%20found%20in-,Basic%20description,limit%20or%20a%20score%20threshold\)..](https://en.wikipedia.org/wiki/Tabu_search#:~:text=be%20found%20in-,Basic%20description,limit%20or%20a%20score%20threshold)..)
- [9] R. TARJAN. “Depth-first search and linear graph algorithms”. In : *SIAM J. Comput* 1 (1972), p. 146-160. URL : https://www.academia.edu/6429212/DEPTH_FIRST_SEARCH_AND_LINEAR_GRAPH_ALGORITHMS.
- [10] Anne de VRIES. “Algorithms for the Edge Orientation Problem”. In : *Master Thesis Operations Research and Quantitative Logistics* (avr. 2022).

Annexe C : Algorithme DFS orientation

Algorithm 8: DFS_Orientation($M, seen, sommet, tree, low, E, prefixe, \vec{E}$)

Result: Orientation fortement connexe de G

```

1   $prefixe[sommet] \leftarrow \text{taille}(prefixe);$ 
2   $low[sommet] \leftarrow prefixe[sommet];$ 
3  foreach  $i \in \Gamma^+(sommet)$  do
4      if  $i \notin seen$  then
5           $tree \leftarrow tree \cup \{(sommet, i)\};$ 
6           $seen \leftarrow seen \cup \{i\};$ 
7          if  $DFS\_Orientation(M, seen, i, tree, low, E, prefixe, \vec{E}) == 0$  then
8              return 0
9          end
10     end
11 end
12  $parent \leftarrow -1;$ 
13 foreach  $i \in \Gamma(sommet)$  do
14     if  $(sommet, i) \notin tree$  and  $(i, sommet) \notin tree$  then
15          $low[sommet] \leftarrow \min(low[sommet], low[i]);$ 
16         if  $\{sommet, i\} \in E$  then
17              $M[i][sommet] \leftarrow 0;$ 
18              $\vec{E} \leftarrow \vec{E} \cup \{(sommet, i)\};$ 
19         end
20     end
21     else if  $(i, sommet) \in tree$  then
22          $parent \leftarrow i;$ 
23     end
24 end
25 if  $parent \neq -1$  then
26     if  $low[sommet] < prefixe[sommet]$  then
27          $low[parent] \leftarrow \min(low[sommet], low[parent]);$ 
28         if  $\{sommet, parent\} \in E$  then
29              $M[sommet][parent] \leftarrow 0;$ 
30              $\vec{E} \leftarrow \vec{E} \cup \{(parent, sommet)\};$ 
31         end
32     end
33     else if  $low[sommet] == prefixe[sommet]$  then
34          $low[sommet] \leftarrow low[parent];$ 
35         if  $\{sommet, parent\} \in E$  then
36              $M[parent][sommet] \leftarrow 0;$ 
37              $\vec{E} \leftarrow \vec{E} \cup \{(sommet, parent)\};$ 
38         end
39     else
40         Erreur : le graphe n'est pas orientable;
41         return 0;
42     end
43 end
44 end
45 return 1;

```

La fonction `taille` renvoie le nombre de clés d'un dictionnaire.

Annexe D : Minimiser le Max

Ici nous allons minimiser le plus grand des plus court chemin, nommé diamètre. Cependant, en réfléchissant, nous avons décidé de minimiser le plus court chemin qui augmente le plus après modification au lieu de juste minimiser le diamètre. Car le diamètre pourrait ne pas être impacté par l'orientation du graphe. Concrètement cela revient à minimiser la plus grande augmentation au lieu de minimiser le plus grand des plus court chemin.

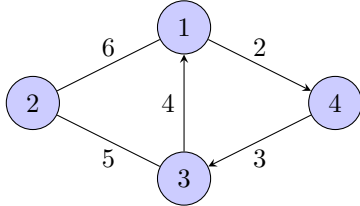
En effet, Chvátal et Thomassen [3] ont étudié l'effet de l'orientation sur le diamètre d'un graphe et ont montré que dans un graphe 2-connexe avec un diamètre d , on peut orienter le graphe de telle sorte que le diamètre du graphe orienté est $\leq 2d^2 + d$.

D'autre part, le diamètre du graphe va correspondre, bien souvent, à la distance entre deux sommets aux extrémités du graphe. En pratique, ils sont déjà loin l'un de l'autre. Donc en minimisant l'augmentation maximum cela impactera plus de gens que seulement minimiser le diamètre.

Dans le modèle suivant l'objectif de minimiser l'augmentation maximum est défini par la contrainte (9).

$$\left\{ \begin{array}{ll} \min & u \\ \text{s.c.} & \\ \sum_{j \in \Gamma^+(i)} x_{ij}^{st} - \sum_{j \in \Gamma^-(i)} x_{ji}^{st} = 0 & \forall i \in V, \forall st \in P, s \neq i, t \neq i \\ \sum_{j \in \Gamma^+(s)} x_{sj}^{st} - \sum_{j \in \Gamma^-(s)} x_{js}^{st} = 1 & \forall st \in P \\ \sum_{j \in \Gamma^+(t)} x_{tj}^{st} - \sum_{j \in \Gamma^-(t)} x_{jt}^{st} = -1 & \forall s, t \in P \\ x_{ij}^{st} \leq u_{ij} & \forall (i, j) \in \vec{E}, st \in P \\ u_{ij} + u_{ji} = 1 & \forall \{i, j\} \in E \\ x_{ij}^{st} \in [0; 1] & \forall (i, j) \in A \\ u_{ij} \in \{0; 1\} & \forall (i, j) \in \vec{E} \\ u \geq \sum_{(i,j) \in A} (d_{ij} \cdot x_{ij}^{st}) - \lambda_{st} & \forall s, t \in V, s \neq t \end{array} \right. \quad \begin{array}{l} (1) \\ (2) \\ (3) \\ (4) \\ (5) \\ (6) \\ (7) \\ (8) \\ (9) \end{array}$$

Voici un exemple :



$P = \{(1, 2), (2, 1), (2, 3), (3, 2), (4, 2)\}$ car les plus courts chemins des couples (1, 3), (3, 1), (1, 4) et (4, 1) ne passent pas par des arêtes.

$$\begin{array}{ll} \lambda_{1,2} = 6 & \lambda_{2,1} = 6 \\ \lambda_{2,3} = 5 & \lambda_{3,2} = 5 \\ \lambda_{2,4} = 6 + 2 = 8 & \lambda_{4,2} = 2 + 6 = 8 \end{array}$$

Il y a deux solutions possibles les autres ne sont pas fortement connexes :

1. $u_{2,1} = 1$ et $u_{3,2} = 1$
2. $u_{1,2} = 1$ et $u_{2,3} = 1$

solution 1 on a :

$$\begin{array}{ll} \mu_{1,2} = 2 + 3 + 5 = 10(+4) & \mu_{2,1} = 6 \\ \mu_{2,3} = 6 + 2 + 3 = 11(+6) & \mu_{3,2} = 5 \\ \mu_{2,4} = 6 + 2 = 8 & \mu_{4,2} = 3 + 5 = 8 \end{array}$$

$$\sum_{st \in P} \sum_{ij \in A} d_{ij} \cdot x_{ij}^{st} = 38$$

augmentation maximum : +6 pour (2, 3)

solution 2 on a :

$$\begin{array}{ll} \mu_{1,2} = 6 & \mu_{2,1} = 5 + 4 = 9(+3) \\ \mu_{2,3} = 5 & \mu_{3,2} = 4 + 6 = 10(+5) \\ \mu_{2,4} = 5 + 3 = 8 & \mu_{4,2} = 2 + 6 = 8 \end{array}$$

$$\sum_{st \in P} \sum_{ij \in A} d_{ij} \cdot x_{ij}^{st} = 46$$

augmentation maximum : +5 pour (3, 2)

Donc la solution optimale est la solution 2, mais si on veut minimiser la somme on remarque que la solution optimale est la solution 1.